МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
„КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ”
НАВЧАЛЬНО-НАУКОВИЙ КОМПЛЕКС
„ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ”

ЛАБОРАТОРНА РОБОТА №3

з курсу: «Інтелектуальні методи діагностики за медичними зображеннями»

Виконав:
студент групи КН-51мп
Сидько Н.С.

КИЇВ
2026

23. Реалізація механізму Early Stopping для запобігання перенавчанню моделі та автоматичного збереження найкращого чекпоінта

Імплементувати механізм Early Stopping у навчальний pipeline U-Net з метою:

1. запобігання перенавчанню (overfitting)
2. оптимізації часу навчання
3. автоматичного збереження найкращої моделі за метрикою валідації

Ключові параметри реалізації

1. patience = 7
2. monitor = validation loss або validation Dice
3. збереження best model (checkpoint)
4. логування моменту зупинки

Теоретичні відомоті

Сегментація медичних зображень є однією з ключових задач медичного комп’ютерного зору. Її суть полягає у тому, щоб для кожного пікселя зображення визначити, чи належить він до області інтересу, наприклад до пухлини, органа або іншої патологічної структури. На відміну від звичайної класифікації, де модель видає лише один клас для всього зображення, у задачі сегментації формується повна карта маски, що точно відображає локалізацію потрібного об’єкта.

Однією з найвідоміших архітектур для сегментації є U Net. Її особливість полягає у симетричній будові, яка складається з енкодера та декодера. Енкодер поступово зменшує просторові розміри зображення та витягує ознаки різних рівнів абстракції. Декодер, навпаки, відновлює просторову роздільну здатність та формує підсумкову карту сегментації. Важливою перевагою U Net є наявність skip connections, які передають детальну просторову інформацію з енкодера до декодера. Саме завдяки цьому модель здатна добре відновлювати межі об’єкта та працювати навіть на відносно невеликих медичних датасетах.

У медичних задачах особливе значення має правильна підготовка даних. Для кожного зображення обов’язково повинна існувати відповідна маска. Якщо порядок файлів порушено, назви не збігаються або маска не відповідає зображенню, модель починає навчатися на некоректних прикладах. Це призводить до деградації метрик, нестабільного навчання і хибних висновків щодо якості моделі. Саме тому перевірка відповідності image mask pairs є обов’язковим етапом перед початком навчання.

Ще однією важливою складовою є нормалізація інтенсивностей зображення. Значення пікселів у медичних зображеннях можуть істотно відрізнятися за діапазоном, тому приведення їх до інтервалу від 0 до 1 дозволяє

стабілізувати процес оптимізації. Крім того, маски для бінарної сегментації повинні бути приведені до формату 0 та 1. Це необхідно для коректного обчислення функції втрат і метрик якості.

Для оцінювання якості сегментації широко застосовується коефіцієнт Dice. Він відображає ступінь перекриття між передбаченою моделлю маскою та еталонною маскою. Чим ближче значення Dice до 1, тим краще модель виділяє цільову область. Значення, близькі до 0, означають, що модель майже не знаходить об’єкт або визначає його неправильно.

Під час навчання глибоких нейронних мереж часто виникає проблема перенавчання. Вона проявляється у тому, що на тренувальній вибірці помилка продовжує зменшуватися, але на валідаційній вибірці якість більше не покращується або навіть погіршується. Це означає, що модель надто сильно підлаштувалася під тренувальні дані та втрачає здатність узагальнювати нові приклади.

Хід роботи

1. Встановлення (імпорт) бібліотек

На першому етапі було підготовлено програмне середовище Google Colab для виконання лабораторної роботи. Для цього було встановлено бібліотеки, необхідні для побудови нейронної мережі, роботи з тензорами, зчитування зображень, візуалізації графіків та підготовки даних. Використання цих бібліотек забезпечило можливість реалізувати повний pipeline сегментації від завантаження датасету до навчання моделі та оцінювання її якості.

```
import os
import random
import numpy as np
import matplotlib.pyplot as plt
import cv2

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, random_split
```

2. Завантаження та розпакування датасету

На наступному етапі було завантажено підготовлений архів із медичним датасетом у середовище Google Colab та виконано його розпакування. Усередині архіву було сформовано дві основні папки: images і masks. Така структура є зручною для задач сегментації, оскільки дозволяє однозначно співставляти кожному зображенню відповідну маску.

У цій роботі було використано датасет, у якому файли зображень мали формат .tif, а маски мали аналогічні назви з додаванням суфікса _mask. Такий формат зберігання даних є придатним для задачі бінарної медичної сегментації та дозволяє легко організувати автоматичне формування пар image mask.

```
import zipfile
import os

zip_path = "/content/mydataset.zip"

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall("/content")

print("Архів розпаковано")

# перевірка
print("\nВміст /content:")
print(os.listdir("/content"))

print("\nВміст images:")
print(os.listdir("/content/images")[:5])

print("\nВміст masks:")
print(os.listdir("/content/masks")[:5])
```

```
Архів розпаковано

Вміст /content:
['.config', 'masks', 'images', 'mydataset.zip', 'sample_data']

Вміст images:
['TCGA_CS_4941_19960909_19.tif', 'TCGA_CS_4941_19960909_3.tif', 'TCGA_CS_4941_19960909_6.tif', 'TCGA_CS_4941_19960909_15.tif', 'TCGA_CS_4941_

Вміст masks:
['TCGA_CS_4941_19960909_23_mask.tif', 'TCGA_CS_4941_19960909_14_mask.tif', 'TCGA_CS_4941_19960909_2_mask.tif', 'TCGA_CS_4941_19960909_4_mask.
```

3. Перевірка відповідності image mask pairs

Після завантаження датасету було виконано перевірку коректності відповідності між зображеннями та масками. Для цього списки файлів було відсортовано, підраховано їх кількість, а також перевірено, що для кожного зображення існує відповідна маска з такою самою базовою назвою та суфіксом _mask.

Цей етап є критично важливим у задачах сегментації. Якщо порядок файлів порушений або маски не відповідають зображенням, модель буде отримувати некоректні приклади навчання. У такому випадку навіть при технічно правильному коді результати моделі будуть хибними.

```
print("\n Усі перевірені пари коректні")
```

```
Зображень: 23
Масок: 23

 Усі перевірені пари коректні
```

4. Створення класу Dataset і попередня обробка даних

Далі було реалізовано власний клас SegmentationDataset, який відповідає за зчитування зображень та масок із диска, приведення їх до єдиного розміру, нормалізацію та підготовку до подачі в модель. На цьому етапі зображення були перетворені у відтінки сірого, масштабовані до розміру 256 на 256, а їх інтенсивності нормалізовані до діапазону від 0 до 1.

Маски також були приведені до розміру 256 на 256, але з використанням nearest interpolation, щоб уникнути розмиття меж. Після цього було виконано їх бінаризацію, у результаті чого маски містили лише значення 0 та 1. Це забезпечило коректність обчислення функції втрат і метрик у задачі бінарної сегментації.

```python
import os
import cv2
import numpy as np
import torch
from torch.utils.data import Dataset

class SegmentationDataset(Dataset):
    def __init__(self, images_dir, masks_dir, image_files, target_size=(256, 256)):
        self.images_dir = images_dir
        self.masks_dir = masks_dir
        self.image_files = image_files
        self.target_size = target_size

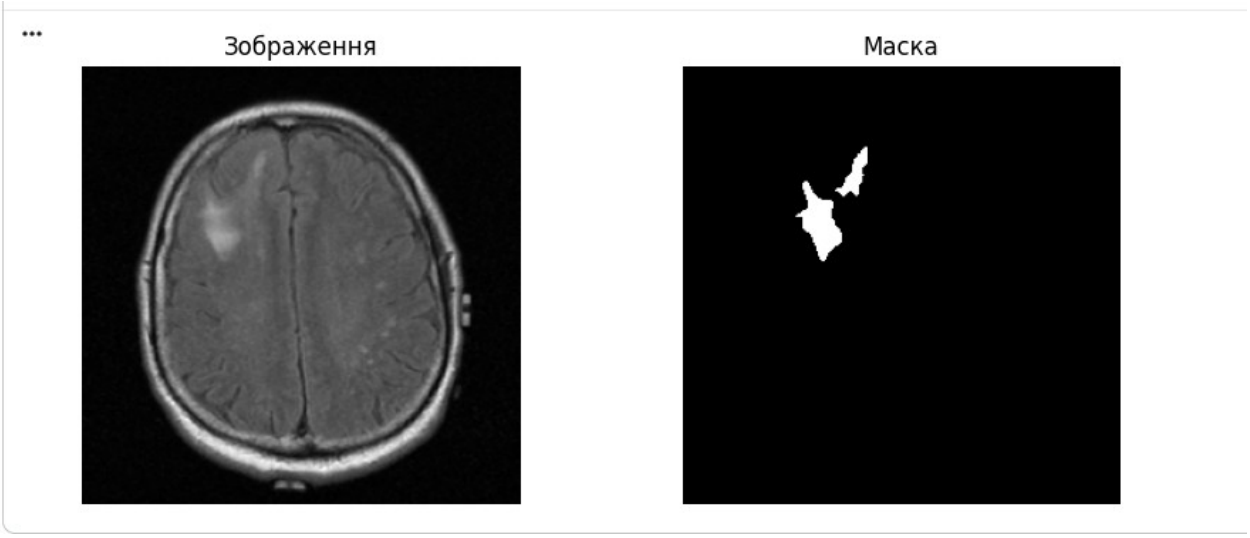
    def __len__(self):
        return len(self.image_files)

    def __getitem__(self, idx):
        img_name = self.image_files[idx]
        mask_name = img_name.replace(".tif", "_mask.tif")
```

5. Перевірка одного елемента датасету та візуальна перевірка

Після створення Dataset було перевірено один приклад із датасету. Було підтверджено, що зображення і маска мають правильну форму 1 x 256 x 256, тип float32, а значення маски містять лише 0 та 1. Додатково було виконано візуальну перевірку зображення та відповідної маски Цей етап дозволив пересвідчитися, що підготовка даних виконана без помилок. Візуальна перевірка є дуже важливою, оскільки дозволяє не лише побачити правильність роботи коду, а й переконатися, що маска справді відповідає цільовій області на медичному зображенні.

```
Розмір датасету: 23
Форма зображення: torch.Size([1, 256, 256])
Форма маски: torch.Size([1, 256, 256])
Тип зображення: torch.float32
Тип маски: torch.float32
Мінімум зображення: 0.0
Максимум зображення: 1.0
Унікальні значення маски: tensor([0.])
```



Результати перевірки показали, що дані мають правильний формат і готові до використання у нейронній мережі. Візуальне співставлення підтвердило, що маска відповідає області інтересу на зображенні.

6. Розбиття датасету на train validation test

Після підготовки даних було виконано розбиття датасету на тренувальну, валідаційну та тестову вибірки у співвідношенні 70 15 15. Для забезпечення відтворюваності результатів було зафіксовано значення випадкового зерна. Такий підхід дозволяє оцінити модель чесно. Тренувальна вибірка використовується для оновлення ваг, валідаційна для контролю якості під час навчання та роботи механізму Early Stopping, а тестова для фінальної незалежної оцінки після завершення навчання.

```
# фіксація випадковості
seed = 42
random.seed(seed)
np.random.seed(seed)
torch.manual_seed(seed)
torch.cuda.manual_seed_all(seed)

# розміри вибірок
dataset_size = len(dataset)
train_size = int(0.70 * dataset_size)
val_size = int(0.15 * dataset_size)
test_size = dataset_size - train_size - val_size

train_dataset, val_dataset, test_dataset = random_split(
    dataset,
    [train_size, val_size, test_size],
    generator=torch.Generator().manual_seed(seed)
)

print("Загальний розмір датасету:", dataset_size)
print("Train:", len(train_dataset))
print("Validation:", len(val_dataset))
print("Test:", len(test_dataset))
```

```
Загальний розмір датасету: 23
Train: 16
Validation: 3
Test: 4
```

7. Створення DataLoader

Для ефективного подавання даних у модель були створені об’єкти DataLoader для train, validation і test вибірок. Було обрано розмір батчу 4, що дозволило подавати дані порціями та оптимізувати процес навчання. Використання shuffle=True для train вибірки дозволило випадково перемішувати дані, що покращує узагальнювальну здатність моделі. Для validation і test вибірок перемішування не застосовувалося, оскільки ці вибірки використовуються для стабільного контролю якості.

```
Кількість train batch: 4
Кількість val batch: 1
Кількість test batch: 1
```

Результат показав, що DataLoader працює коректно та формує батчі для всіх трьох вибірок. Це означає, що дані підготовлено до безпосереднього навчання моделі.

8. Перевірка одного batch із DataLoader

Перед побудовою моделі було виконано перевірку одного батчу з DataLoader. Було підтверджено, що батч зображень і батч масок мають форму batch_size x 1 x 256 x 256, а значення масок залишаються бінарними. Цей етап дозволяє остаточно переконатися, що pipeline підготовки даних працює правильно. Якщо на цьому етапі форма або тип даних були б неправильними, модель U Net не змогла б коректно працювати.

```python
images, masks = next(iter(train_loader))

print("Форма batch зображень:", images.shape)
print("Форма batch масок:", masks.shape)

print("Тип зображень:", images.dtype)
print("Тип масок:", masks.dtype)

print("Мін значення зображень:", images.min().item())
print("Макс значення зображень:", images.max().item())

print("Унікальні значення масок:", torch.unique(masks))
```

```
Форма batch зображень: torch.Size([4, 1, 256, 256])
Форма batch масок: torch.Size([4, 1, 256, 256])
Тип зображень: torch.float32
Тип масок: torch.float32
Мін значення зображень: 0.0
Макс значення зображень: 1.0
Унікальні значення масок: tensor([0.])
```

9. Побудова архітектури U Net

На цьому етапі було реалізовано базову 4 рівневу архітектуру U Net. Модель включала блоки подвійних згорток, шари зменшення розміру через MaxPool2d, bottleneck блок, шари ConvTranspose2d для апсемплінгу та skip connections для передачі просторової інформації між відповідними рівнями енкодера та декодера. Після побудови моделі було виконано перевірку forward pass. Було встановлено, що модель приймає батч вхідних даних форми B x 1 x 256 x 256 і повертає вихід такої ж просторової розмірності. Це підтвердило правильність реалізації архітектури.

```python
class DoubleConv(nn.Module):
    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1),
            nn.ReLU(inplace=True)
        )

    def forward(self, x):
        return self.block(x)


class UNet(nn.Module):
    def __init__(self, in_channels=1, out_channels=1):
        super().__init__()
```

```python
    def forward(self, x):
        e1 = self.enc1(x)
        e2 = self.enc2(self.pool1(e1))
        e3 = self.enc3(self.pool2(e2))
        e4 = self.enc4(self.pool3(e3))

        b = self.bottleneck(self.pool4(e4))
```

10. Визначення функції втрат, оптимізатора та метрики

Після побудови моделі було визначено функцію втрат BCEWithLogitsLoss, оптимізатор Adam та метрику Dice. Функція втрат використовувалася для оцінювання помилки моделі під час навчання, а Dice застосовувався для контролю якості сегментації на валідаційних та тестових даних.

Спочатку під час експерименту спостерігалася ситуація, коли значення validation Dice залишалося рівним нулю, хоча validation loss зменшувався. Це було пов’язано з жорстким порогуванням прогнозів. Після корекції реалізації метрики було отримано адекватні значення Dice, що дозволило коректно відслідковувати якість сегментації в процесі навчання.

```python
import torch.nn as nn
import torch.optim as optim

# функція втрат
criterion = nn.BCEWithLogitsLoss()

# оптимізатор
optimizer = optim.Adam(model.parameters(), lr=1e-3)

# Dice метрика
def dice_score(preds, targets, smooth=1e-6):
    preds = torch.sigmoid(preds)

    intersection = (preds * targets).sum()
    union = preds.sum() + targets.sum()

    dice = (2. * intersection + smooth) / (union + smooth)
    return dice
```

11. Навчання моделі з використанням Early Stopping

Відповідно до індивідуального варіанту 23 було реалізовано механізм Early Stopping. Для навчання було задано максимальну кількість епох, параметр patience дорівнював 7, а моніторинг здійснювався за значенням validation loss. У разі покращення validation loss автоматично зберігався checkpoint найкращої моделі. Якщо покращення не відбувалося протягом семи епох поспіль, навчання завершувалося автоматично.

У ході навчання спостерігалося поступове зменшення train loss і validation loss. Значення validation Dice зросло від приблизно 0.22 на початкових етапах до приблизно 0.87 наприкінці навчання. Це свідчить про те, що модель не тільки успішно оптимізувалася, але й поступово навчилася коректно виділяти область інтересу на валідаційних даних.

Навчання було автоматично зупинено після 41 епохи, оскільки validation loss протягом заданої кількості епох більше не покращувався. Це підтвердило правильність реалізації механізму Early Stopping та його практичну ефективність для запобігання перенавчанню і скорочення часу обчислень.

```
Без покращення: 4/7
Epoch 39/50
Train Loss: 0.0093 | Val Loss: 0.0065 | Val Dice: 0.8715
Без покращення: 5/7
Epoch 40/50
Train Loss: 0.0087 | Val Loss: 0.0070 | Val Dice: 0.8542
Без покращення: 6/7
Epoch 41/50
Train Loss: 0.0083 | Val Loss: 0.0073 | Val Dice: 0.8433
Без покращення: 7/7
Early Stopping: навчання зупинено
```

12. Завантаження найкращої моделі та оцінка на test

Після завершення навчання було завантажено найкращий збережений checkpoint моделі та виконано оцінювання на тестовій вибірці. У результаті було отримано Test Loss = 0.0081 та Test Dice = 0.6987.

Отримані результати свідчать про те, що модель зберегла здатність узагальнювати дані та може коректно працювати на нових зображеннях, які не використовувалися під час навчання. Значення Dice на тестовій вибірці є нижчим, ніж на validation, що є природним для невеликого датасету, але водночас залишається достатньо високим для підтвердження працездатності моделі.

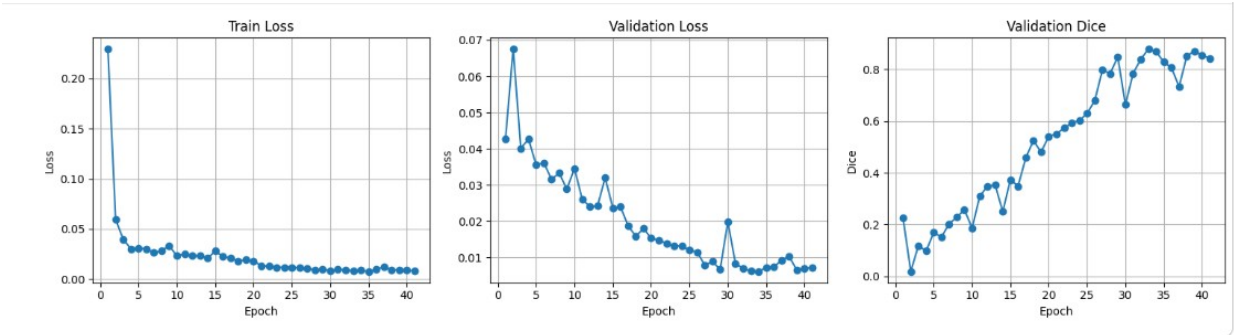
```
print(f"Test Loss: {test_loss:.4f}")
print(f"Test Dice: {test_dice:.4f}")
```

```
Test Loss: 0.0081
Test Dice: 0.6987
```

13. Побудова графіків збіжності

На завершальному етапі було побудовано графіки train loss, validation loss та validation Dice. Аналіз графіків показав, що train loss швидко зменшувався на початку навчання, а потім поступово виходив на стабільний рівень. Validation loss також загалом зменшувався, хоча на окремих епохах спостерігалися незначні коливання, що є нормальним для невеликого датасету.

Найбільш показовим є графік validation Dice. На ньому видно чітке зростання метрики від низьких значень на початку навчання до високих значень наприкінці, що свідчить про ефективне формування сегментаційної маски моделлю. Також графіки підтверджують, що механізм Early Stopping спрацював після досягнення стабілізації якості.



Висновок

У ході виконання лабораторної роботи було реалізовано повний pipeline сегментації медичних зображень на основі архітектури U Net. Було виконано підготовку датасету у форматі image mask pairs, перевірено коректність відповідності між зображеннями та масками, здійснено нормалізацію вхідних даних і бінаризацію масок. Далі було сформовано train, validation і test вибірки, побудовано DataLoader та реалізовано базову 4 рівневу архітектуру U Net.

Особлива увага в роботі була приділена реалізації індивідуального варіанту 23, а саме механізму Early Stopping. У процесі навчання контролювався validation loss, при його покращенні автоматично зберігався найкращий checkpoint моделі, а за відсутності покращення протягом 7 епох навчання завершувалося автоматично. Це дозволило запобігти перенавчанню моделі та зменшити витрати часу на навчання.

У результаті навчання було досягнуто високого значення validation Dice, яке зросло приблизно до 0.87, а на тестовій вибірці було отримано Test Dice = 0.6987 при Test Loss = 0.0081. Отримані результати свідчать про ефективність реалізованої моделі та правильність обраного підходу. Отже, мету лабораторної роботи досягнуто, а механізм Early Stopping підтвердив свою практичну доцільність у задачі медичної сегментації.

Посилання на Google Colab:
https://colab.research.google.com/drive/132WS7DJheg6wuAOslmlJvlF9Xl7eOdk4?usp=sharing